



Universidade do Minho
Escola de Engenharia

Computação Gráfica

Trabalho Prático: Fase 1

Licenciatura em Engenharia Informática
Ano Letivo 2025/2026

Grupo 03

a106919 Eduardo Freitas Fernandes
a106842 Gonçalo Rodrigues Ribeiro
a106888 José Mário Raimundo Lima

Índice

1	Introdução	2
2	Primitivas Geométricas	3
2.1	Plano	3
2.2	Cubo	4
2.3	Esfera	5
2.4	Cone	7
3	Primitivas Geométricas Extras	11
3.1	Cilindro	11
3.2	Anel	12
4	Arquitetura do Sistema	14
4.1	Formato do Ficheiro 3D	15
4.2	Generator	15
4.3	Engine	16
4.4	Controlo da Câmara	16
5	Conclusão	17

1 Introdução

O presente relatório insere-se no âmbito da unidade curricular de **Computação Gráfica** e descreve o desenvolvimento da primeira fase do projeto, composta por duas aplicações distintas: o *Generator* e o *Engine*. Esta fase tem como objetivo principal a construção de um sistema capaz de gerar e visualizar **primitivas geométricas** tridimensionais de forma modular e estruturada.

A aplicação **Generator** é responsável pela geração de modelos geométricos, nomeadamente planos, caixas, esferas e cones (e cilindros e anéis). A partir de um conjunto de parâmetros fornecidos na linha de comandos, esta aplicação calcula os vértices correspondentes a cada primitiva e exporta-os para ficheiros num formato próprio (.3d). Estes ficheiros constituem uma representação intermédia da geometria, funcionando como elemento de ligação entre a fase de geração e a fase de visualização.

Por sua vez, a aplicação **Engine** tem como função interpretar ficheiros de configuração em formato XML, onde se encontra descrita a cena a renderizar. Com base nessa descrição, o *Engine* procede ao carregamento dos modelos previamente gerados e realiza a sua renderização através da biblioteca **OpenGL**. Esta separação entre geração e renderização assegura uma organização clara de responsabilidades e estabelece uma base sólida para a extensão do sistema nas fases seguintes do projeto.

A implementação foi realizada na linguagem **C++**, recorrendo à OpenGL para a visualização gráfica e a um parser XML para a leitura da descrição da cena. Ao longo deste relatório são apresentadas as principais **decisões** de projeto adotadas, a **arquitetura do sistema** e os **algoritmos** utilizados na geração das primitivas geométricas.

2 Primitivas Geométricas

2.1 Plano

Matematicamente, um plano pode ser definido por um ponto de origem P_0 e dois vetores diretores não colineares, $\vec{u}$ e $\vec{v}$. Qualquer ponto P pertencente ao plano é obtido através da combinação linear:

$$P(s, t) = P_0 + s \times \vec{u} + t \times \vec{v}$$

Onde $s, t \in [0, \text{length}]$ representam os parâmetros escalares de distância ao longo dos eixos locais da face. Para que o plano se apresente centrado na origem do sistema de coordenadas global, o ponto de partida P_0 é deslocado metade da dimensão total em sentido oposto aos vetores diretores. No plano XZ , com a normal orientada segundo o eixo Y , a configuração inicial estabelece:

$$P_0 = \left(-\frac{L}{2}, 0, \frac{L}{2}\right), \quad \vec{u} = (1, 0, 0), \quad \vec{v} = (0, 0, -1)$$

Para a representação digital, a superfície contínua é dividida numa malha de $n \times n$ células quadrangulares. O passo de discretização, definido por $\Delta = \frac{L}{n}$, determina a resolução da malha. Cada célula (i, j) no domínio discreto, onde $i, j \in \{0, \dots, n-1\}$, possui quatro vértices canônicos definidos pelas coordenadas locais:

1. Vértice do canto esquerdo inferior: $V_{bl} = P_0 + (i \times \Delta) \times \vec{u} + (j \times \Delta) \times \vec{v}$
2. Vértice do canto direito inferior: $V_{br} = P_0 + \Delta \times \vec{u}$
3. Vértice do canto esquerdo superior: $V_{tl} = P_0 + \Delta \times \vec{v}$
4. Vértice do canto direito superior: $V_{tr} = P_0 + \Delta \times \vec{u} + \Delta \times \vec{v}$

As figuras 1 e 2 representam as duas primeiras iterações do algoritmo.

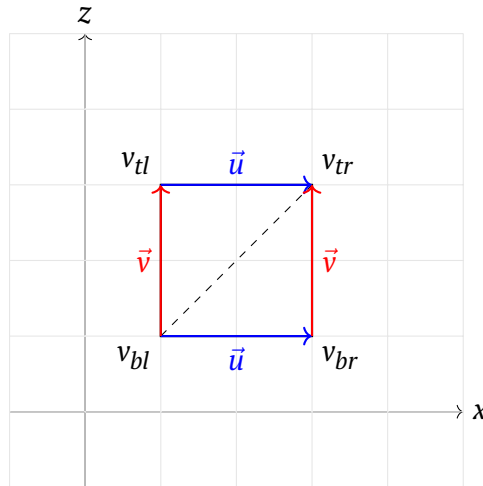


Figura 1: Triangulação de uma célula do plano (1ª iteração)

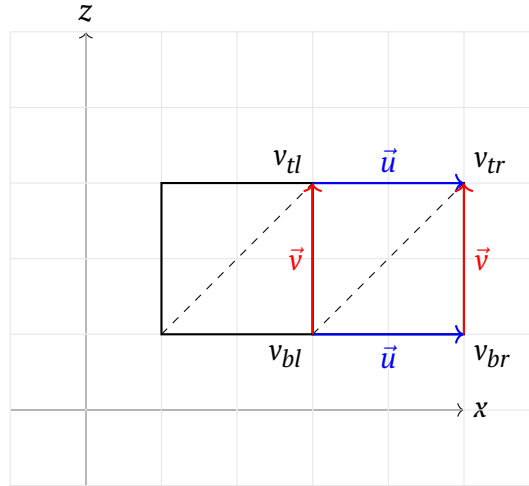


Figura 2: Triangulação de uma célula do plano (2ª iteração)

A estabilidade geométrica da malha requer que cada quadrícula seja subdividida em dois triângulos. A triangulação ocorre através da partição da célula por uma diagonal, tipicamente unindo V_{br} a V_{tl} , resultando nos triângulos $T_1 = \{V_{bl}, V_{br}, V_{tl}\}$ and $T_2 = \{V_{br}, V_{tr}, V_{tl}\}$.

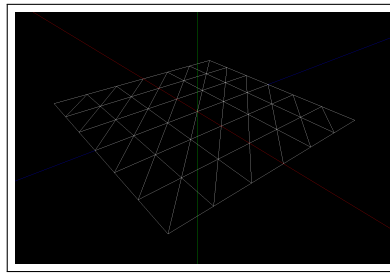


Figura 3: Plano com 3 unidades de comprimento e 6 divisões

2.2 Cubo

O procedimento de criação de um cubo utiliza a lógica de construção de planos discutida anteriormente, onde cada face é mapeada para um dos planos cartesianos principais ou para planos paralelos a estes, deslocados de uma distância $\frac{d}{2}$ em relação à origem, onde d representa a dimensão da aresta (representado por dimension no código).

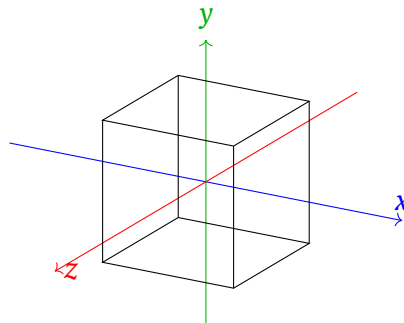


Figura 4: Representação do cubo centrado na origem

Para garantir que a face do sólido está virada para a frente, cada face é definida por uma origem específica P_0 e dois vetores diretores $\vec{u}$ e $\vec{v}$. As seis faces são agrupadas em pares opostos ao longo dos eixos X , Y e Z (seja $\vec{x}$, $\vec{y}$, $\vec{z}$ os vetores elementares dos eixos):

- **Eixo Z (+Z e -Z):** As faces frontal e traseira são geradas no plano XY . A face em $+Z$ utiliza os vetores $\{\vec{x}, \vec{y}\}$, enquanto a face em $-Z$ inverte o sentido de $\vec{u}$ para $\{-\vec{x}, \vec{y}\}$, compensando a orientação para manter a consistência da malha.
- **Eixo X (+X e -X):** As faces laterais são definidas no plano ZY . A transição de orientação é feita trocando os componentes dos vetores diretores, utilizando o eixo Z (positivo ou negativo) como direção horizontal local e o eixo Y como vertical.
- **Eixo Y (+Y e -Y):** As faces superior e inferior fecham a caixa no plano XZ . Aqui, as faces utilizam combinações de $\vec{x}$ e $\vec{z}$, garantindo que o topo e a base do cubo sejam perfeitamente ortogonais às faces laterais.

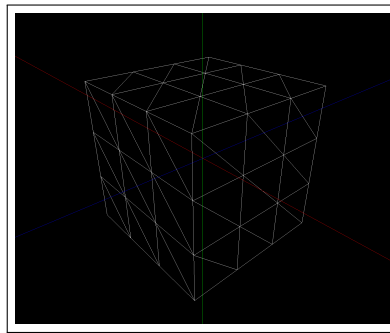


Figura 5: Caixa com 2 de dimensão e 3 divisões

2.3 Esfera

A construção geométrica de uma esfera em computação gráfica baseia-se na sua parametrização matemática, seguida da discretização da superfície em elementos planos (triângulos). O objetivo é aproximar uma superfície contínua por uma malha poligonal suficientemente fina. Considere-se uma esfera de raio r , centrada na origem do sistema cartesiano tridimensional. A sua definição implícita é:

$$x^2 + y^2 + z^2 = r^2$$

A esfera pode ser descrita através de coordenadas esféricas usando dois parâmetros angulares:

- $\theta \in [0, \pi]$ - corresponde à variação vertical
- $\phi \in [0, 2\pi]$ - corresponde à variação horizontal

A conversão para coordenadas cartesianas é dada por:

$$x = r \times \sin(\theta) \times \sin(\phi)$$

$$y = r \times \cos(\theta)$$

$$z = r \times \sin(\theta) \times \cos(\phi)$$

Por outras palavras, o ângulo θ controla a altura na esfera, para cada valor fixo de θ obtemos um círculo horizontal com raio $r_\theta = r \times \sin(\theta)$ e altura $y = r \times \cos(\theta)$. O ângulo ϕ percorre esse círculo.

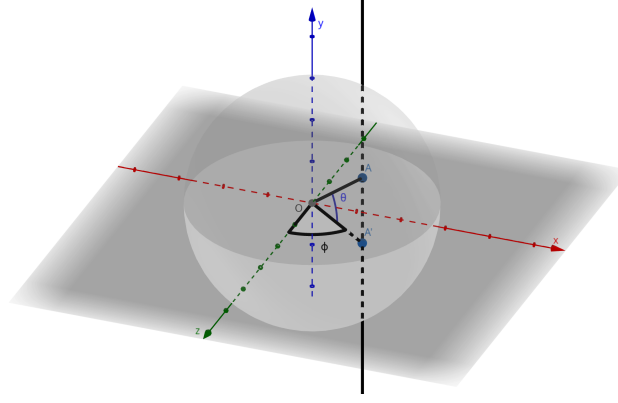


Figura 6: Representação tridimensional da esfera

Para aproximar numericamente a esfera, os intervalos angulares são subdivididos:

$$\Delta\theta = \frac{\pi}{\text{slices}}$$

$$\Delta\phi = \frac{2\pi}{\text{stacks}}$$

Assim:

$$\theta_i = i \times \Delta\theta, \quad i = 0, 1, \dots, \text{slices}$$

$$\phi_j = j \times \Delta\phi, \quad j = 0, 1, \dots, \text{stacks}$$

Esta discretização cria uma grelha no domínio angular (θ, ϕ) . Cada célula da malha corresponde a um pequeno retângulo no plano dos parâmetros:

$$[\theta_i, \theta_{i+1}] \times [\phi_j, \phi_{j+1}]$$

Definimos então os seguintes 4 ângulos que delimitam uma região retangular no espaço:

$$\theta_1 = \theta_i$$

$$\theta_2 = \theta_{i+1}$$

$$\phi_1 = \phi_j$$

$$\phi_2 = \phi_{j+1}$$

Cada par (θ, ϕ) gera um ponto na esfera, obtendo no total 4 pontos que formam um retângulo (dois triângulos):

$$P_1 = \begin{pmatrix} r \times \sin \theta_1 \times \sin \phi_1 \\ r \times \cos \theta_1 \\ r \times \sin \theta_1 \times \cos \phi_1 \end{pmatrix}$$

$$P_2 = \begin{pmatrix} r \times \sin \theta_1 \times \sin \phi_2 \\ r \times \cos \theta_1 \\ r \times \sin \theta_1 \times \cos \phi_2 \end{pmatrix}$$

$$P_3 = \begin{pmatrix} r \times \sin \theta_2 \times \sin \phi_1 \\ r \times \cos \theta_2 \\ r \times \sin \theta_2 \times \cos \phi_1 \end{pmatrix}$$

$$P_4 = \begin{pmatrix} r \times \sin \theta_2 \times \sin \phi_2 \\ r \times \cos \theta_2 \\ r \times \sin \theta_2 \times \cos \phi_2 \end{pmatrix}$$

Estes quatro pontos formam um quadrilátero curvo na superfície da esfera. Uma vez que o OpenGL trabalha com triângulos, cada quadrilátero é dividido em dois triângulos:

$$T_1 = (P_1, P_4, P_2)$$

$$T_2 = (P_1, P_3, P_4)$$

O algoritmo para criar uma esfera consiste em:

1. Percorrer sucessivamente valores de θ , criando círculos horizontais.
2. Em cada círculo, percorrer valores de ϕ , criando pontos com o mesmo espaçamento.
3. Ligar pontos adjacentes entre dois círculos consecutivos.
4. Dividir cada região em dois triângulos.

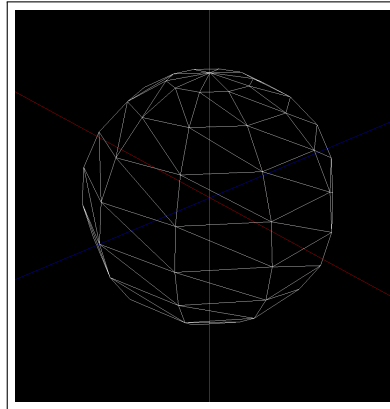


Figura 7: Esfera com 1 de raio, 10 slices e 10 stacks

2.4 Cone

Um cone reto pode ser descrito como um sólido cuja base é um círculo de raio r num plano horizontal e cujo vértice está alinhado verticalmente acima do centro da base, a uma altura h . Assumimos aqui um cone com:

- base no plano $y = 0$
- centro da base na origem $(0, 0, 0)$
- eixo do cone alinhado com o eixo y
- vértice em $(0, h, 0)$

A base é o conjunto de pontos que satisfaz:

$$y = 0$$

$$x^2 + z^2 = r^2$$

Uma parametrização do círculo da base usa o ângulo $\theta \in [0, 2\pi]$, que é o ângulo ao redor do eixo y , gera as seguintes coordenadas cartesianas (visível na figura 8):

- $x = r \times \cos(\theta)$
- $z = r \times \sin(\theta)$
- $y = 0$

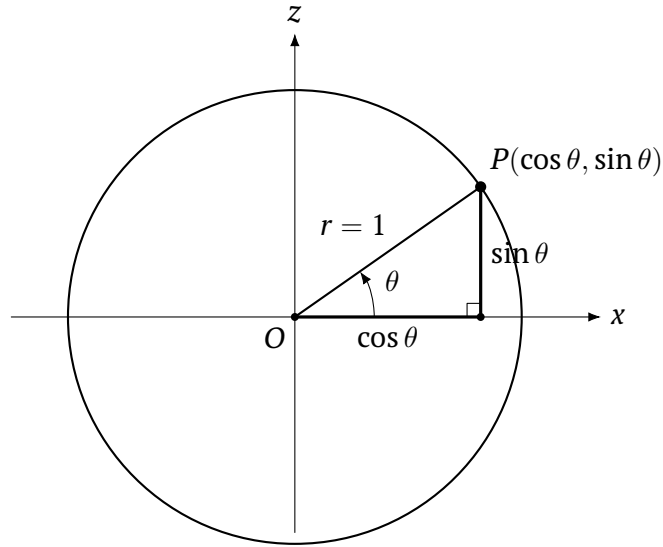


Figura 8: Coordenadas polares no plano cartesiano

A face lateral pode ser vista como a união de círculos horizontais à medida que subimos em y . A ideia essencial é à altura l , o raio do cone é menor do que r , esse raio diminui até zero ao chegar ao vértice do cone. Se definirmos um parâmetro de altura normalizado:

$$t = \frac{y}{h} \Rightarrow t \in [0, 1]$$

então o raio da secção circular à altura l é:

$$R(l) = r \times (1 - t) = r \times \left(1 - \frac{l}{h}\right)$$

garantindo que quando $l = 0 \Rightarrow R(0) = r$ que corresponde á base e quando $l = h \Rightarrow R(h) = 0$ que corresponde ao vértice do cone. O cone é parametrizado em duas componentes:

- **slices:** divisões ao redor do cone (em ângulo)
- **stacks:** divisões ao longo da altura

Tal como na esfera teremos:

$$\Delta_\theta = \frac{2\pi}{\text{slices}}$$

Para cada índice j :

$$\theta_j = j \times \Delta_\theta$$

$$\theta_{j+1} = (j + 1) \times \Delta_\theta$$

Estes dois ângulos definem as duas bordas laterais de uma fatia do cone.

$$\Delta t = \frac{1}{\text{stacks}}$$

Para cada índice i :

$$t_i = \frac{i}{\text{stacks}}$$

$$t_{i+1} = \frac{i+1}{\text{stacks}}$$

e as alturas correspondentes são:

$$l_i = t_i \times h$$

$$l_{i+1} = t_{i+1} \times h$$

Os raios correspondentes são:

$$R_i = r \times (1 - t_i)$$

$$R_{i+1} = r \times (1 - t_{i+1})$$

Isto cria círculos horizontais ao longo do cone. Ao triangularem a base, escolhem-se dois ângulos consecutivos:

$$\theta_0 = \theta_j = \frac{2 \times \pi \times j}{\text{slices}}$$

$$\theta_1 = \theta_{j+1} = \frac{2 \times \pi \times (j+1)}{\text{slices}}$$

Com isso, criam-se dois pontos no círculo da base:

$$P_0 = (r \times \cos \theta_0, 0, r \times \sin \theta_0)$$

$$P_1 = (r \times \cos \theta_1, 0, r \times \sin \theta_1)$$

E usa-se também o centro:

$$C = (0, 0, 0)$$

Cada fatia da base é então um triângulo:

$$(C, P_0, P_1)$$

Repetindo este procedimento para todos os j , obtém-se um conjunto de triângulos que preenche a base. A superfície lateral usa a mesma ideia: para cada altura l_i (stack) e ângulo θ_j (slice), define-se um ponto num círculo de raio R_i ao nível l_i :

- $x_{i,j} = R_i \times \cos \theta_j$
- $y_{i,j} = l_i$
- $z_{i,j} = R_i \times \sin \theta_j$

onde temos:

- $l_i = \frac{i \times h}{\text{stacks}}$
- $R_i = r \times \left(1 - \frac{i}{\text{stacks}}\right)$
- $\theta_j = \frac{2 \times \pi \times j}{\text{slices}}$

Este é o procedimento geométrico por trás da função que calcula pontos do cone (`conePoint()`). Entre dois níveis consecutivos i e $i+1$, e entre dois ângulos consecutivos j e $j+1$, temos quatro pontos:

- $V_{bl} = S(t_i, \theta_j)$ (inferior-esquerdo)
- $V_{br} = S(t_i, \theta_{j+1})$ (inferior-direito)
- $V_{tl} = S(t_{i+1}, \theta_j)$ (superior-esquerdo)
- $V_{tr} = S(t_{i+1}, \theta_{j+1})$ (superior-direito)

Considerando as fórmulas:

$$S(t, \theta) = \begin{pmatrix} r \times (1 - t) \times \cos \theta \\ h \times t \\ r \times (1 - t) \times \sin \theta \end{pmatrix}$$

$$t = \frac{i}{\text{stacks}}, \quad i = 0, \dots, \text{stacks}$$

$$\theta = \frac{2 \times \pi \times j}{\text{slices}}, \quad j = 0, \dots, \text{slices}$$

Esses quatro pontos formam uma célula lateral, que é dividida em 2 triângulos:

$$(V_{bl}, V_{tl}, V_{tr})$$

$$(V_{bl}, V_{tr}, V_{br})$$

Ao repetir para todos os i e j , obtém-se a superfície lateral completa.

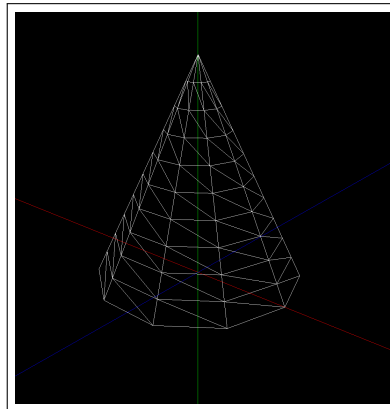


Figura 9: Cone com 1 de raio, 2 de altura, 10 slices e 10 stacks

3 Primitivas Geométricas Extras

3.1 Cilindro

A construção de um cilindro circular reto baseia-se na descrição geométrica de duas superfícies planas (bases) e de uma superfície lateral desenvolvível. Considera-se um cilindro de raio r e altura h , centrado na origem do sistema de coordenadas, com eixo principal coincidente com o eixo y . Neste caso, a base superior encontra-se no plano $y = \frac{h}{2}$ e a base inferior no plano $y = -\frac{h}{2}$. Matematicamente, o cilindro pode ser descrito como o conjunto de pontos que satisfazem:

$$x^2 + z^2 = r^2 \quad \text{com} \quad y \in \left[-\frac{h}{2}, \frac{h}{2}\right].$$

A geração da malha parte da discretização angular da circunferência da base. O intervalo angular completo é $[0, 2\pi]$, o qual é dividido uniformemente em um número fixo de partes designado por *slices*. Define-se o incremento angular como:

$$\Delta_\alpha = \frac{2 \times \pi}{\text{slices}}.$$

Para cada índice inteiro (i), considera-se:

$$\alpha_i = i \times \Delta_\alpha \quad \text{e} \quad \alpha_{i+1} = (i + 1) \times \Delta_\alpha.$$

Estes dois ângulos consecutivos determinam dois pontos adjacentes na circunferência de raio r :

$$P_i = (r \times \cos \alpha_i, y, r \times \sin \alpha_i),$$

$$P_{i+1} = (r \times \cos \alpha_{i+1}, y, r \times \sin \alpha_{i+1}).$$

Para a base superior fixa-se $y = \frac{h}{2}$, enquanto para a base inferior fixa-se $y = -\frac{h}{2}$. A triangulação da base superior é realizada considerando o centro do disco:

$$C_{top} = (0, \frac{h}{2}, 0),$$

e formando triângulos do tipo:

$$(C_{top}, P_{i+1}, P_i).$$

Este procedimento é repetido para todos os valores de i , resultando numa decomposição radial do disco em triângulos que partilham um vértice comum no centro. De modo análogo, a base inferior é triangulada usando o ponto descrito abaixo, garantindo o preenchimento completo do disco inferior.

$$C_{bottom} = (0, -\frac{h}{2}, 0),$$

A superfície lateral é construída a partir da observação de que o cilindro pode ser entendido como a translação vertical de uma circunferência de raio constante. Para cada par angular consecutivo (α_i, α_{i+1}) , definem-se quatro pontos:

$$P_i^- = (r \times \cos \alpha_i, -\frac{h}{2}, r \times \sin \alpha_i),$$

$$P_{i+1}^- = (r \times \cos \alpha_{i+1}, -\frac{h}{2}, r \times \sin \alpha_{i+1}),$$

$$P_i^+ = (r \times \cos \alpha_i, \frac{h}{2}, r \times \sin \alpha_i),$$

$$P_{i+1}^+ = (r \times \cos \alpha_{i+1}, \frac{h}{2}, r \times \sin \alpha_{i+1}).$$

Estes quatro pontos definem um quadrilátero na superfície lateral do cilindro. Como a representação gráfica utiliza exclusivamente triângulos, esse quadrilátero é dividido em dois triângulos adjacentes:

$$(P_{i+1}^-, P_i^-, P_{i+1}^+)$$

$$(P_{i+1}^+, P_i^-, P_i^+).$$

A repetição deste processo para todos os valores de i produz uma malha lateral composta por faixas retangulares trianguladas. Do ponto de vista geométrico, o método corresponde à discretização da parametrização:

$$S(\alpha, y) = \begin{pmatrix} r \times \cos \alpha \\ y \\ r \times \sin \alpha \end{pmatrix}, \quad \alpha \in [0, 2\pi], \quad y \in \left[-\frac{h}{2}, \frac{h}{2}\right].$$

A discretização é efetuada apenas na variável angular, enquanto a coordenada vertical assume apenas dois valores extremos para cada faixa lateral. O resultado é uma aproximação poligonal do cilindro cuja qualidade aumenta proporcionalmente ao número de slices. Quanto maior o número de divisões angulares, menor será a diferença entre a malha gerada e a superfície cilíndrica contínua.

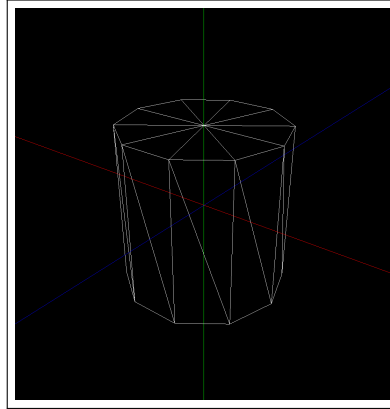


Figura 10: Cilindro com 1 de raio, 2 de altura e 10 slices

3.2 Anel

A função `generateRing()` constrói geometricamente um anel tridimensional. Esta superfície pode ser entendida como o resultado da rotação de um círculo em torno de um eixo exterior ao próprio círculo. Mais concretamente, considera-se um círculo de raio r (designado por raio menor) cujo centro percorre uma circunferência de raio R (designado por raio maior). O eixo de rotação é perpendicular ao plano do círculo gerador.

Geometricamente, o processo pode ser descrito em duas etapas conceptuais. Primeiro, define-se um círculo de raio r num plano vertical. Em seguida, esse círculo é deslocado de modo que o seu centro fique a uma distância R do eixo vertical. Finalmente, todo o conjunto é rodado em torno desse eixo, descrevendo uma trajetória circular completa.

A descrição matemática utiliza dois ângulos independentes. O primeiro ângulo, usualmente representado por θ , controla a rotação do círculo gerador em torno do eixo principal. O segundo ângulo, ϕ , percorre os pontos do próprio círculo gerador. A parametrização clássica do anel pode ser escrita, de forma compacta, como:

- $x = (R + r \times \cos \phi) \times \cos \theta$

- $y = r \times \sin \phi$
- $z = (R + r \times \cos \phi) \times \sin \theta$

O termo $R + r \times \cos \phi$ representa a distância radial ao eixo vertical. Quando $\phi = 0$, o ponto encontra-se na parte exterior do anel, quando $\phi = \pi$, encontra-se na parte interior. O valor de y é determinado exclusivamente pela posição no círculo gerador.

Para gerar a malha poligonal, o domínio angular completo de ambos os ângulos é dividido uniformemente. O ângulo θ é subdividido em um certo número de rings, que correspondem às divisões ao longo da direção circular principal do anel. O ângulo ϕ é subdividido em sides, que correspondem às divisões ao longo da secção transversal do anel.

Cada par de valores consecutivos (θ_i, θ_{i+1}) e (ϕ_j, ϕ_{j+1}) define uma pequena região retangular no espaço paramétrico. No espaço tridimensional, essa região transforma-se num quadrilátero curvo sobre a superfície do toro. Os quatro pontos correspondentes são obtidos aplicando a parametrização aos quatro pares de ângulos combinados.

Como a representação gráfica utiliza triângulos, cada quadrilátero é dividido em duas faces triangulares. A ordenação consistente dos vértices garante que estes estão virados para a "frente", garantindo a sua visualização.

Em síntese, o procedimento baseia-se na rotação de um círculo de raio r ao longo de uma trajetória circular de raio R , seguida da discretização uniforme dos dois parâmetros angulares que descrevem essa construção.

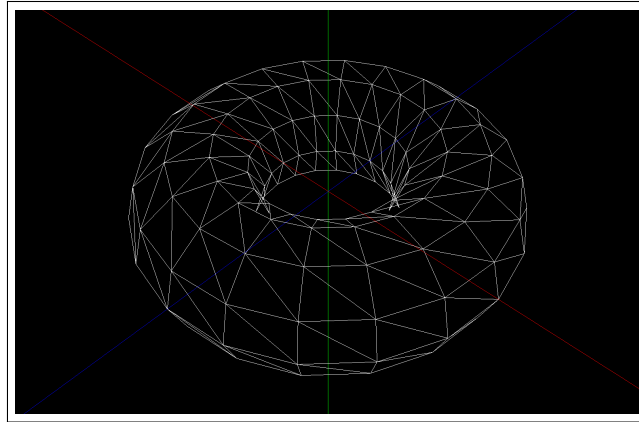


Figura 11: Anél com 2 de raio maior, 1 de raio menor, 10 sides e 20 rings

4 Arquitetura do Sistema

O sistema desenvolvido nesta fase segue uma arquitetura modular composta por duas aplicações principais: **Generator** e **Engine**. O **Generator** é responsável por produzir malhas triangulares de primitivas geométricas a partir de parâmetros fornecidos na linha de comandos, exportando essa geometria para ficheiros .3d. A **Engine** interpreta um ficheiro de configuração em **XML**, carrega os modelos referenciados e procede à sua renderização através de **OpenGL**.

Esta organização estabelece uma separação clara entre a geração procedural (cálculo de vértices e triangulação) e a fase de visualização, promovendo modularidade, reutilização de código e facilidade de extensão para fases futuras.

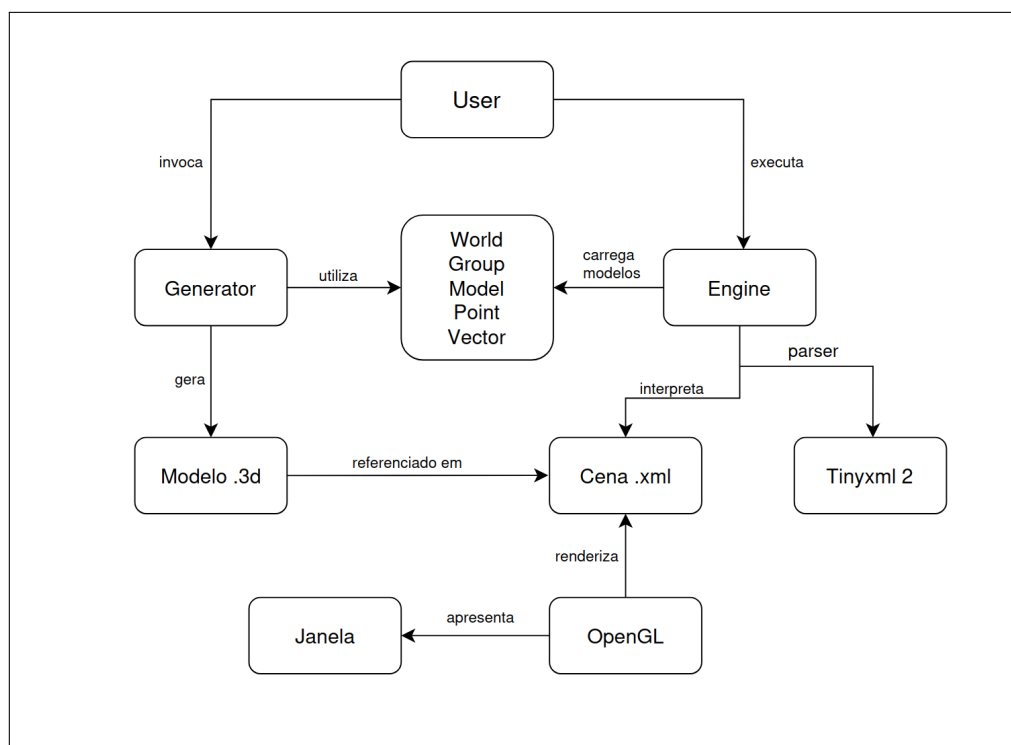


Figura 12: Esquema geral da Arquitetura do Sistema

Como ilustrado na Figura 12, a arquitetura encontra-se estruturada em dois módulos independentes que partilham um conjunto de estruturas internas comuns (World, Group, Model, Point e Vector). Esta camada intermédia assegura uma representação consistente da geometria e da organização da cena, permitindo que o *Generator* e a *Engine* reutilizem as mesmas abstrações sem dependências diretas entre si.

4.1 Formato do Ficheiro 3D

O ficheiro `.3d` constitui a representação intermédia da geometria gerada pelo **Generator**, servindo como ligação entre a fase de geração e a fase de visualização. O seu objetivo é armazenar, de forma simples e estruturada, os vértices que compõem a malha triangulada de cada primitiva.

```
540 ← n° triângulos
triângulo { 0 1 0
            0.25 0.951057 0.181636
            0.309017 0.951057 0
triângulo { 0 1 0
            0.0954915 0.951057 0.293893
            0.25 0.951057 0.181636
triângulo { 0 1 0
            -0.0954915 0.951057 0.293893
            0.0954915 0.951057 0.293893
            0 1 0
            -0.25 0.951057 0.181636
            -0.0954915 0.951057 0.293893
            0 1 0
            -0.309017 0.951057 -2.70151e-08
            -0.25 0.951057 0.181636
            0 1 0
            -0.25 0.951057 -0.181636
            -0.309017 0.951057 -2.70151e-08
```

Figura 13: Formato do ficheiro 3D

Como se observa na Figura 13, a primeira linha indica o número total de vértices armazenados no ficheiro. As restantes linhas correspondem às coordenadas tridimensionais de cada vértice, organizadas sequencialmente no formato `x y z`. Cada conjunto consecutivo de três vértices define um triângulo da malha, permitindo que o **Engine** reconstrua a geometria diretamente a partir da ordem de leitura. Esta organização dos valores permite um parsing simples.

4.2 Generator

O **Generator** é uma aplicação de linha de comandos que gera primitivas geométricas e exporta o resultado para ficheiros `.3d`. Nesta fase foram implementadas as primitivas: plano, caixa, esfera e cone. Adicionalmente, foram desenvolvidas duas primitivas extra: cilindro e anel. Cada comando recebe os parâmetros geométricos necessários (dimensão/raio/altura, divisões, slices e stacks) e o nome do ficheiro de saída.

Internamente, os vértices são acumulados numa estrutura **Model**, através da adição sequencial de triângulos. No caso da caixa, foi utilizado um procedimento genérico de construção de faces parametrizado por um ponto de origem e dois vetores direção, permitindo gerar as seis faces com o mesmo algoritmo e reduzindo duplicação de código. Por fim, a função `toFile()` serializa os vértices para o formato `.3d`.

4.3 Engine

A **Engine** é responsável por carregar e renderizar uma cena descrita num ficheiro **XML**. O **XML** contém a configuração da janela e da câmara, bem como a lista de modelos **.3d** a carregar.

A interpretação do **XML** é efetuada com a biblioteca **tinyxml2**, sendo os valores extraídos armazenados em estruturas próprias do projeto (por exemplo, classes como **World**, **Group** e **Model**). Após a leitura do ficheiro, a **Engine** carrega os ficheiros **.3d**, reconstruindo a lista de vértices. A renderização é realizada em OpenGL, desenhando-se cada triângulo a partir de conjuntos consecutivos de três vértices.

4.4 Controlo da Câmara

Além da renderização dos modelos, a aplicação *Engine* integra um sistema de câmara interativa baseado em coordenadas esféricas, centrada na origem do mundo. As coordenadas cartesianas iniciais, extraídas do ficheiro XML de configuração, são convertidas nos parâmetros r (raio), α e β (ângulos), permitindo uma navegação orbital fluida em torno da cena.

Esta conversão facilita a rotação em torno do centro da cena. As variáveis esféricas consistem no raio de visão (r) e nos ângulos horizontal (α) e vertical (β). O cálculo dinâmico da nova posição da câmara a cada frame é feito através das seguintes equações:

$$x = r \times \cos(\beta) \times \sin(\alpha)$$

$$y = r \times \sin(\beta)$$

$$z = r \times \cos(\beta) \times \cos(\alpha)$$

A interação do utilizador é gerida através de *callbacks* da biblioteca GLUT:

- **Teclas Direcionais (Setas):** Alteram os ângulos α e β , permitindo rodar a câmara em torno da cena. O ângulo β foi limitado ao intervalo $[-\frac{\pi}{2}, \frac{\pi}{2}]$ radianos para evitar a inversão da câmara.
- **Teclas 'w' e 's':** Alteram o raio (r), aproximando (zoom in) ou afastando (zoom out) a câmara da origem (com um limite mínimo de 1.0).
- **Tecla 'l':** Alterna o estado de bloqueio da câmara (*fixedCam*), permitindo fixar a perspetiva ou libertá-la para exploração livre.

Sempre que os parâmetros são alterados, a função `glutPostRedisplay()` é invocada, atualizando a janela e alimentando os novos valores cartesianos à função `gluLookAt`.

5 Conclusão

A primeira fase deste projeto prático permitiu concretizar com sucesso os objetivos propostos relativos à geração procedimental e renderização gráfica de primitivas 3D.

A separação do sistema em duas aplicações independentes, *Generator* e *Engine*, demonstrou ser uma abordagem estrutural adequada. O *Generator* assegura o cálculo matemático dos vértices das diferentes figuras (plano, caixa, esfera e cone), efetuando a sua serialização para o formato intermédio ".3d". Por sua vez, o *Engine* assume o processamento do ficheiro de configuração XML, com recurso à biblioteca *tinyxml2*, e a consequente renderização da cena através do OpenGL. A integração de uma câmara orbital interativa revelou-se um mecanismo fundamental nesta fase, permitindo a exploração espacial e a validação da correção geométrica dos modelos gerados.

Durante o desenvolvimento, a principal exigência técnica centrou-se no cálculo das coordenadas esféricas e polares para a divisão da esfera e do cone em *slices* e *stacks*. Para a construção da *box*, adotou-se uma estratégia de reaproveitamento da lógica algorítmica do plano, aplicando as devidas transformações espaciais para formar as seis faces, o que permitiu otimizar a estrutura do código.